Finance App — Project Plan

Oct 14, 2026 · @Afonso

Overview

A multi-user browser app for budgeting and investment tracking, built over roughly five to six months as a pilot to grow design, architecture and implementation skills on AWS.

Primary goal: personal skill growth. Features are kept when they are technically instructive, even if a minimal product would skip them.

Secondary goal: a product you actually use, with an architecture that could become commercial without a rewrite.

What it does

- Tracks money in a double-entry ledger, filled by manual entry and CSV import.
- Shows portfolio holdings, returns and allocation against your targets.
- Runs budgets and works out a monthly investable surplus.
- Checks rules you define and turns them into suggestions with an explanation.
- Presents a Google News-style feed: subscribed topics, thesis-threat alerts and rule recommendations.

Non-goals for the pilot

- No bank or broker connections. Everything is entered manually or imported from CSV.
- No native mobile app and no offline mode. Responsive web only.
- No predictions or buy and sell advice generated by the app.
- No payments or billing.

Decisions so far

These are settled and shape everything below.

Decision	Choice	What it rules out
Client	Desktop-first browser app, responsive on phones	Native apps, offline mode and sync for now
Users	Multi-user from day one, with shared workspaces	Retrofitting tenancy later

Decision	Choice	What it rules out
Data entry	Manual entry and CSV import only	Bank (PSD2) and broker API connections
Suggestions	Checks on rules the user defines, with explanations	App-generated predictions or trade advice
News	Google News-style feed: topic subscriptions, thesis-threat alerts, rule recommendations	A raw per-holding news firehose
Hosting	Serverless on AWS, eu-central-1, no Route 53	Always-on servers, NAT Gateway, ALB, RDS
Budget	Under $5–10 a month	WAF, interface endpoints, large LLM usage

Product scope

Six areas, each chosen for a distinct technical challenge.

Area	What the user gets	Main technical challenge
Ledger	Accounts, transactions, transfers, splits, net worth	Double-entry model, append-only events, corrections as reversal entries, integer money
CSV import	Column-mapping screen with preview, saved templates per source, undo a whole import	Messy formats (decimal commas, date formats, encodings), duplicate detection on overlapping files, idempotent re-import
Portfolio	Holdings, cost basis, dividends, allocation against targets, returns	Multi-currency, withholding tax, splits, ISIN-to-ticker mapping, time-weighted and money-weighted (XIRR) returns
Budgeting	Categories, budgets, recurring payments, monthly investable surplus	Categorisation that learns from corrections, recurring-payment detection
Rules and suggestions	User-defined rules and explained suggestions	A small rule language, scheduled and event-driven evaluation, an explanation trail

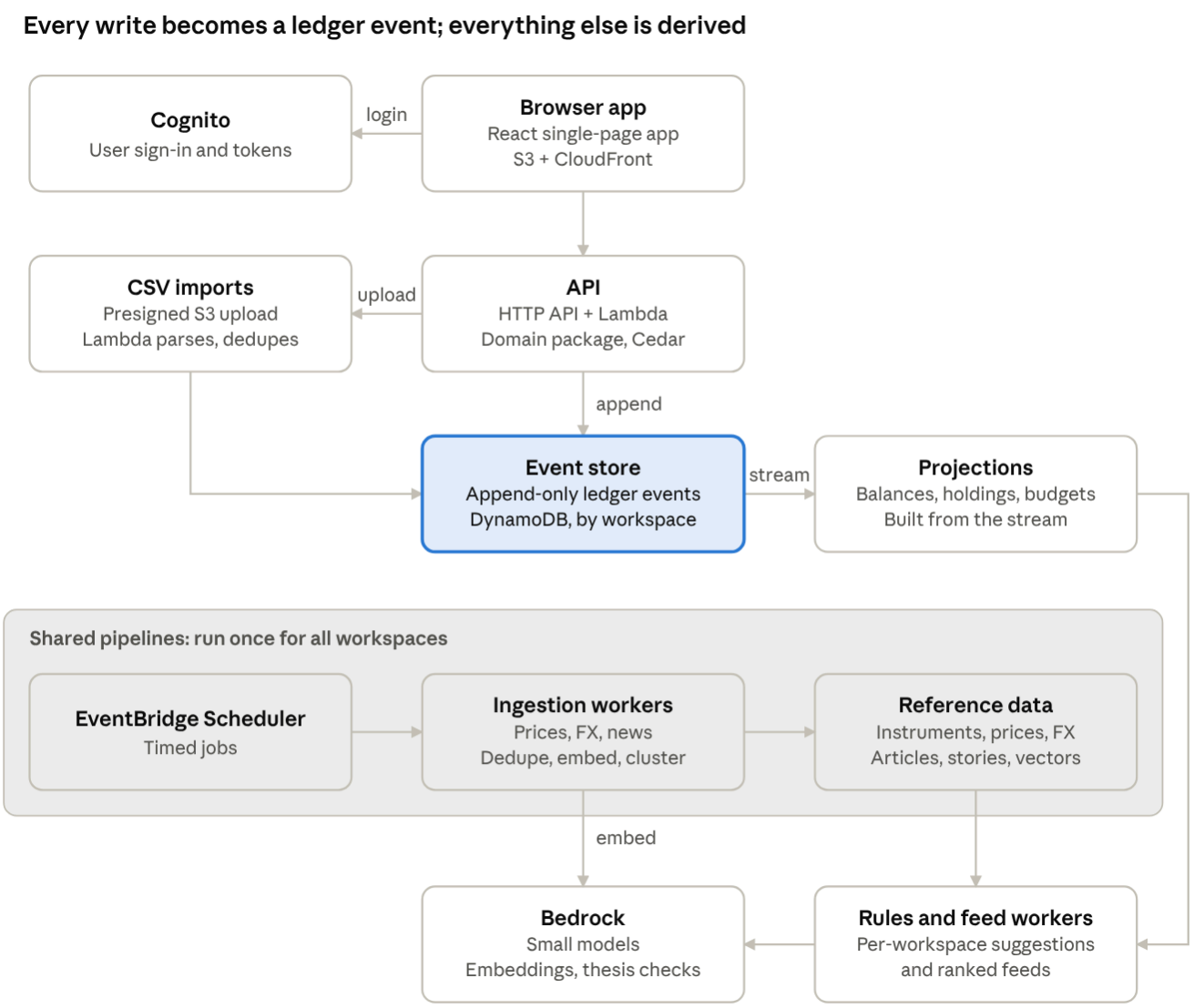
Area	What the user gets	Main technical challenge
News feed	Topic subscriptions, story clusters, thesis-threat alerts, rule cards in one feed	Shared ingestion, story clustering, ranking, LLM classification measured against an evaluation set

Workspaces: every record belongs to a workspace. A user can have a personal workspace and share a household one. Roles are owner, member and viewer, and individual accounts can be private inside a shared workspace.

Theses: a written investment thesis is a first-class record, linked to holdings and topics. It replaces a markdown file, and it drives both thesis-threat alerts and thesis review reminders.

Architecture

Fully serverless with no VPC: the server is the source of truth, ledger events are the core record, and everything else is a projection built from them.



architecture · write path, projections and shared pipelines

The API also reads projections and feeds to serve the browser; those read paths are left out to keep the picture readable.

Components

- **Frontend:** single-page app (React or similar) on S3 + CloudFront, using the default CloudFront domain.
- **Auth:** Cognito user pool; the access token carries user identity only, never workspace permissions.
- **API:** API Gateway HTTP API → Lambda (TypeScript). API style is decided in ADR-003.
- **Domain package:** pure TypeScript with no AWS or browser dependencies (ledger rules, portfolio maths, rule evaluation). Used by Lambdas now, and portable to the browser if offline mode returns.
- **Event store:** ledger events in DynamoDB, keyed by workspace.

- **Projections:** DynamoDB Streams → Lambda builds balances, holdings and budget totals. Projections can be rebuilt from events.
- **Imports:** S3 presigned upload → Lambda parse and dedupe, or parse in the browser (ADR-005).
- **Shared pipelines:** EventBridge Scheduler triggers market-data and news ingestion once for all users.
- **Rules and feed workers:** scheduled and event-driven Lambdas evaluate rules and build each workspace's feed.
- **LLM:** Bedrock with small models for news classification and summaries, behind a budget cap.

Data model principles

- Money stored as integer minor units plus a currency code. Never floats.
- IDs generated by the client (UUIDv7) and an idempotency key on every write, so retries are safe.
- Editable records (budgets, rules, categories) carry a version number; stale writes are rejected with If-Match.
- Shared reference data (instruments, prices, FX, articles) is separate from workspace data (ledger, rules, theses, feeds).

Tenancy and security

- Authorization as Cedar policies evaluated inside the Lambdas, tested on their own.
- Workspace ID leads every DynamoDB key. Each request runs with credentials scoped to that workspace via the `dynamodb:LeadingKeys` IAM condition, so a handler bug cannot read another workspace.
- Encryption at rest everywhere, an audit log of data access, and only minimal data sent to Bedrock.

Cost guardrails

- AWS Budgets alarm at $5 and $10.
- CloudWatch log retention set in CDK from day one.
- Avoid NAT Gateway, ALB, RDS, WAF, interface VPC endpoints and Secrets Manager (use SSM SecureString).
- Shared pipelines cost the same for one user or fifty; LLM calls are capped per day.

News and insights feed

One feed in the Google News style, where articles, thesis threats and rule suggestions all appear as cards ranked for the workspace.

Feed tabs

Tab	What appears	Source
For you	Top-ranked mix of everything below	Ranking across all card types
Thesis watch	Stories that may threaten a written thesis, with the thesis named and a one-line reason	LLM classification of candidate articles against each thesis
Suggestions	Fired rules: rebalancing, contribution allocation, review reminders	Rules engine
Topics	One tab per subscribed topic, e.g. ECB rates, EU energy policy, AI capex	Topic matching on the shared article corpus
Full coverage	One story with every source covering it, opened from any card	Story clustering

Topics and subscriptions

- A topic is a name plus keywords, entities and an embedding. Users subscribe to topics, follow or mute them, and can create their own.
- Holdings and theses automatically become topics, but holding topics are muted by default, so per-stock noise stays out unless it threatens a thesis.
- "Show less like this" and "Not a threat" feedback adjusts ranking and becomes labelled data for the evaluation set.

Pipeline

1. Ingest RSS feeds and open news sources on a schedule, once for all users. Store headline, snippet, link, source and time; never republish full articles.
2. Deduplicate and embed each article once.
3. Cluster articles into stories by embedding similarity within a time window, updated as new articles arrive.
4. Match stories to topics and entities, shared across all workspaces.
5. Cheap pre-filter: only stories that mention a thesis's linked entities or themes go to the LLM.
6. Classify each candidate against the thesis text with a small Bedrock model: threat or not, severity, and a short reason.
7. Build each workspace's feed, merging article cards with rule-suggestion cards.

Ranking signals: subscription match, thesis-threat severity, recency decay, source diversity within a story, and the user's feedback.

Hard parts worth learning

- Building feeds ahead of time (fan-out on write) versus on request (fan-out on read). A good ADR.
- Incremental clustering that stays stable as stories grow.
- An evaluation set for thesis-threat classification, with precision and recall tracked across prompt and model changes.
- Keeping LLM cost bounded: embed once, pre-filter hard, cache results per story and thesis.

Rules engine

Suggestions only ever come from rules the user wrote, and every suggestion shows which rule fired and on what data.

Example rules

Rule type	Example	Evaluated
Rebalancing band	Alert when any ETF drifts more than 5 points from its target weight	After price updates and new trades
Contribution allocation	Put this month's surplus into the asset furthest below target	Monthly, after budget close
Position limit	Alert when a single stock exceeds 5% of the portfolio	After price updates
Thesis review	Remind me to review each thesis quarterly, or when a threat of high severity appears	Scheduled and event-driven
Budget	Alert when a category passes 80% of its monthly budget	On new transactions
Cash buffer	Keep at least 3 months of expenses in cash before investing	Monthly

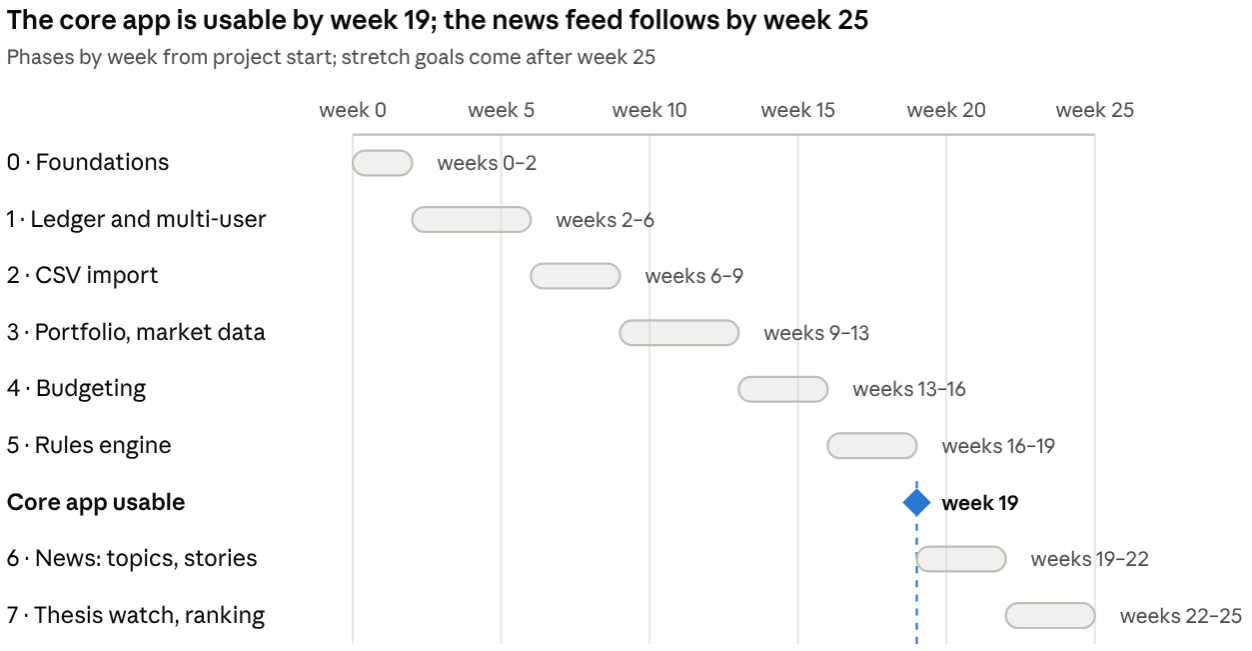
Design

- Rules are stored as structured data, built in a form-based editor first. A text rule language is a stretch goal.

- Rules are evaluated by the domain package against projections, so they are unit-testable without AWS.
- Each evaluation produces a result record with inputs, outcome and explanation. Results are deduplicated so the same condition does not alert daily.
- Fired rules become cards in the Suggestions tab and optionally an email.
- Every rule has a dry-run mode that shows what it would have fired over past data.

Development phases

Eight phases over about 25 weeks, each ending in something deployed and usable, with an exit check before moving on.



phase timeline · weeks 0 to 25

Weeks are a target, not a promise: a phase ends when its exit check passes.

Phase 0 – Foundations (2 weeks)

- Deliver: monorepo with domain package, CDK stacks, GitLab CI with OIDC into dev and prod accounts, Cognito, the frontend on CloudFront, Budgets alarms, structured logging and tracing.
- Write ADR-001 to ADR-004 before writing feature code.
- Exit: a signed-in "hello world" deployed to prod entirely through the pipeline.
- Learn: project setup, ADR discipline, the CI/CD baseline.

Phase 1 – Ledger and multi-user (4 weeks)

- Deliver: double-entry event store, projections via DynamoDB Streams, manual entry screens, workspaces, invites, roles, Cedar policies, IAM-scoped isolation.
- Exit: property-based tests prove every transaction balances; a cross-workspace read is denied by IAM even when the handler is wrong.
- Learn: event sourcing, projections and rebuilds, policy-based authorization, tenant isolation.

Phase 2 – CSV import (3 weeks)

- Deliver: mapping screen with preview, saved templates, duplicate detection, undo of a whole import. Templates for DeGiro, Trading 212 and two banks.
- Exit: importing the same file twice changes nothing; overlapping exports produce no duplicates.
- Learn: idempotency, data pipelines, defensive parsing, golden-file tests.

Phase 3 – Portfolio and market data (4 weeks)

- Deliver: shared instrument, price and FX pipeline; holdings, cost basis, dividends and withholding tax, allocation against targets, time-weighted and money-weighted returns.
- Exit: values reconcile with broker statements within a small tolerance.
- Learn: external data behind adapters, caching, financial maths, shared versus tenant data.

Phase 4 – Budgeting (3 weeks)

- Deliver: categories and budgets, categorisation rules with a learning fallback, recurring-payment detection, the monthly investable surplus.
- Exit: a full month of real transactions categorised with few manual fixes.
- Learn: feedback loops, simple ML or LLM classification, time-series reasoning.

Phase 5 – Rules engine (3 weeks)

- Deliver: rule editor, scheduled and event-driven evaluation, explanations, dry-run, deduplicated alerts, Suggestions tab and email.
- Exit: every rule in the examples table works and explains itself.
- Learn: rule-engine design, scheduling, notification design.

Phase 6 – News feed: ingestion, topics and stories (3 weeks)

- Deliver: shared ingestion, deduplication, embeddings, story clustering, topic subscriptions, Topics and Full coverage tabs.
- Exit: a daily feed of clustered stories for your subscribed topics, under the cost cap.
- Learn: ingestion pipelines, vector search, clustering, fan-out design.

Phase 7 – Thesis watch and ranking (3 weeks)

- Deliver: thesis records, pre-filter, Bedrock classification, Thesis watch tab, ranked For you tab, feedback loop.
- Exit: an evaluation set of at least 100 labelled article and thesis pairs, with precision and recall reported for each prompt or model change.
- Learn: LLM evaluation, ranking, cost control.

Phase 8 – Stretch goals

- Live updates between household members over WebSockets.
- Offline mode and a mobile-friendly installable app.
- Scenarios and projections (Monte Carlo, relocation scenario, Dutch Box 3 estimate).
- A text rule language.
- A read-only chat assistant that queries your data through tools.

ADR backlog

ADR-001 to ADR-004 are decided in Phase 0; the rest are decided at the start of the phase that needs them.

ADR	Question	Options to weigh	Decide by
ADR-001	Where does data live?	DynamoDB single-table; Aurora Serverless v2 scaling to zero via the Data API; DynamoDB for writes plus Parquet on S3 queried with DuckDB or Athena	Phase 0
ADR-002	Tenancy and isolation model	Workspace-prefixed keys with IAM LeadingKeys; per-workspace tables; application-only checks	Phase 0
ADR-003	API style	REST with OpenAPI and generated TypeScript types; GraphQL with AppSync	Phase 0
ADR-004	Repository and deployment layout	Monorepo with packages for domain, api, web, infra; CDK stack split; environments	Phase 0
ADR-005	Where CSV files are parsed	In the browser; S3 upload → Lambda pipeline; both	Phase 2

ADR	Question	Options to weigh	Decide by
ADR-006	Event schema versioning	Upcasting old events on read; versioned projections; migration by replay	Phase 1
ADR-007	Market data providers	Which free-tier providers for prices and FX, behind an adapter	Phase 3
ADR-008	Rule representation	Structured JSON rules; an expression language; Cedar-like policies	Phase 5
ADR-009	Feed generation	Fan-out on write; fan-out on read; hybrid	Phase 6
ADR-010	Vector storage	S3 Vectors; DynamoDB plus in-Lambda search; a small local index in S3	Phase 6

Each ADR is one page: context, options, decision, consequences, and what would make you revisit it.

Working method

You own the thinking; Claude Code owns most of the typing. The split below is what turns vibe coding into skill growth.

You write, before prompting

- The ADR for any decision in the backlog.
- The domain model and the DynamoDB access-pattern table for each phase.
- The invariants and key tests, especially for the ledger, imports and returns maths.
- The exit check for the phase.

Claude Code does

- Implementation against your tests and ADRs, in small merge requests.
- Boilerplate: CDK constructs, API handlers, UI screens.
- First drafts of test fixtures and golden files, which you review.

Habits

- Review every diff as if it were a colleague's merge request; reject code you cannot explain.

- Keep a CLAUDE.md in the repo with the architecture summary, conventions and the current phase.
- Track work in Jira or GitLab issues per phase; close a phase only when its exit check passes.
- Write a short retro at the end of each phase: what you learned, what you would decide differently.

Risks and open questions

Risks

Risk	Impact	Mitigation
Scope creep across eight phases	Project stalls in month three	Hard exit checks; stretch goals stay in Phase 8
Free market-data or news sources change terms	Pipelines break	Adapters per provider; cache everything; ADR-007 lists a fallback
LLM costs grow with usage	Budget exceeded	Pre-filter, per-story caching, daily call cap, small models
Returns maths subtly wrong	Numbers you cannot trust	Reconcile against broker statements in Phase 3; property-based tests
Vibe-coded code you do not understand	Little skill gained	You write ADRs, models and tests first; review every diff

Open questions

- Which two banks' CSV formats to support first in Phase 2?
- Which news sources to start with, and in which languages (English only, or also Dutch and Portuguese)?
- Should email be the only notification channel, or browser push as well?

Regulatory note: in the EU, giving people personalised investment recommendations is regulated under MiFID II. Keeping suggestions strictly to rules the user wrote, with no app-generated advice, keeps the pilot clear of this. Any commercial version would need a proper legal review.